AVANCE 2 — Población, monedas y reportes

Cubre las opciones 3, 4, 5, 6 y 7 de la TP1 del "Sistema de formateo y análisis de datos de países y monedas".

La Opción 3 procesa población, área y densidad. La Opción 4 procesa las monedas y sus tasas. Las Opciones 5, 6 y 7 generan los reportes solicitados por la consigna.

Estado del avance

Commit	Nombre	Estado
4	`feat: implementar analisis poblacional y geografico`	Hecho y verificado
5	`feat: implementar procesamiento de monedas`	Hecho y verificado
6	`feat: generar reportes txt y html`	Hecho y verificado
7	`feat: implementar bitacora del sistema`	Hecho y verificado
8	`refactor: estructurar codigo en camelCase, mejorar documentacion y reportes`	Hecho y verificado
9	`fix: limpiar residuo confirmado en generador html`	Hecho y verificado
10	`docs: documentar verificacion de reportes txt, html y bitacora en avance 2`	Hecho y verificado
11	`docs: documentar auditoria de integracion general y menu principal en avance 2`	Hecho y verificado

Confirmar con `git log --oneline` y `git status` que los commits estén registrados correctamente.

Rama de trabajo: `main`.

Decisiones de diseño

- **Población:** los cálculos de total, promedio y mediana se realizan manualmente, respetando las restricciones de la consigna.
- **Clasificación de población:** se mantienen las categorías indicadas por el PDF: Megaciudad, Ciudad grande, Ciudad mediana y Ciudad pequeña.
- **Densidad:** se calcula a partir de población y área, y se permite obtener los 10 países con mayor y menor densidad.
- **Monedas:** se procesan los códigos, nombres y tasas de cambio sin diccionarios ni funciones automáticas prohibidas.
- **Reportes:** se mantienen como archivos TXT y HTML generados desde el programa de consola.
- **Restricciones:** no se utilizan diccionarios, `.replace()`, `.split()`, `sorted`, `min`, `max`, `sum`, `round`, cortes `[a:b]`, `for`, `lambda` ni comprensiones.
- **Modularización:** se reutilizan las funciones existentes de lectura, normalización y cálculo siempre que corresponde.
- **Validaciones:** los archivos con datos inválidos muestran advertencias y el programa continúa sin cerrarse.

COMMIT 4

Nombre:

```
feat: implementar analisis poblacional y geografico
```

Resultado verificado

- 25 países procesados.
- Población total: **4 411 741 600**.
- Mediana de población: **51 740 000**.
- 21 países clasificados como Megaciudad.
- 4 países clasificados como Ciudad grande.
- Cálculo de área y densidad funcionando.
- Cálculo de los países con mayor y menor densidad funcionando.

COMMIT 5

Nombre:

```
feat: implementar procesamiento de monedas
```

Resultado verificado

- 21 monedas procesadas.
- Tasa promedio: **393.5148**.
- 16 monedas con tasa mayor que 1.
- 2 monedas con tasa igual a 1.
- 3 monedas con tasa menor que 1.
- Procesamiento de códigos y nombres de monedas funcionando.
- Ordenamiento de tasas funcionando.

También se verificó la consulta de países por letra:

- Letra `"a"`: promedio **7**.
- Estados Unidos: **14**.
- Perú: **4**.
- Total: **20 países**.

COMMIT 6

Nombre:

```
feat: generar reportes txt y html
```

Resultado verificado

- El reporte TXT se genera correctamente.
- El reporte HTML de monedas se genera correctamente.
- El reporte HTML de densidad se genera correctamente.
- Los contenidos de los reportes coinciden con los resultados comprobados anteriormente.

- La Opción 7 genera los 10 países con mayor densidad, los 10 con menor densidad y el gráfico ASCII.

COMMIT 7

Nombre:

```
feat: implementar bitacora del sistema
```

Alcance

Implementación de la Opción 8 (bitácora del sistema en `bitacora.bin` con submenú A-E) y registro de todas las acciones del menú principal y errores.

Resultado verificado

- Persistencia binaria mediante `open("bitacora.bin", "ab")` y codificación UTF-8.
- Submenú A (Buscar por fecha), B (Buscar por palabra clave), C (Mostrar todas), D (Exportar a CSV `bitacora.csv`) y E (Salir del submenú).
- Registro del archivo consultado, cantidad de registros y tiempo de respuesta en la Opción 1.
- Registro de salida en la Opción 9.

COMMIT 8

Nombre:

```
refactor: estructurar codigo en camelCase, mejorar documentacion y reportes
```

Alcance

- Estandarización de nombres de funciones y variables en formato camelCase en todos los módulos.
- Mejora de comentarios descriptivos y docstrings en español para cada función.
- Diseño visual mejorado y consistente para los reportes HTML con estilos CSS inline portables.

COMMIT 9

Nombre:

```
fix: limpiar residuo confirmado en generador html
```

Alcance

- Eliminación del residuo de código muerto `html = "自由"` en `construirTablaDetalladaMonedas` de `reportes.py`.
- Verificación de la correcta generación de `reporte_monedas.html` y `reporte_densidad.html`.

Verificación antes del siguiente commit

```
# Revisar usos prohibidos
Select-String -Path *.py -Pattern
"\.replace\(|\.split\(|sorted\(|min\(|max\(|sum\(|round\(|for |lambda|enumerate|zip\("

# Revisar diccionarios
Select-String -Path *.py -Pattern "dict\("

# Revisar estado
git status

# Revisar commits
git log --oneline
```

Verificación general

Se probó el programa completo y las **Opciones 1 a 9** mantienen los resultados esperados y verificados matemáticamente de forma independiente.

Resultados principales

- **Población mundial total:** 4 411 741 600 habitantes.
- **Población promedio:** 176 469 664.00 habitantes.
- **Mediana de población (impar, n=25):** 51 740 000.0 habitantes (Corea del Sur, posición central 12).
- **Mediana de población (par, n=24):** 51 628 281.0 habitantes (promedio de posiciones 11 y 12).
- **Categorías poblacionales:**
  - Megaciudad (> 10M): 21 países.
  - Ciudad grande (1M - 10M): 4 países (Panamá, Uruguay, Costa Rica, Suiza).
  - Ciudad mediana (100k - 999k): 0 países.
  - Ciudad pequeña (< 100k): 0 países.
  - Total clasificados: 25 países.
- **Área total geográfica:** 62 099 758 km^2.
- **Densidad máxima:** Corea Del Sur (516.32 hab/km^2).
- **Densidad mínima:** Australia (3.34 hab/km^2).
- **Monedas únicas analizadas:** 21 monedas.
- **Tasa de cambio promedio:** 393.5148.
- **Moneda mayor tasa (criterio PDF):** Peso Colombiano (COP, 4150.0).
- **Moneda menor tasa (criterio PDF):** Libra Esterlina (GBP, 0.76).
- **Clasificación frente a 1 USD:**
  - Monedas con tasa > 1 USD: 16 monedas.
  - Monedas con tasa = 1 USD: 2 monedas (USD, PAB).
  - Monedas con tasa < 1 USD: 3 monedas (EUR, GBP, CHF).
  - Total monedas clasificadas: 21 monedas.
- **Conteo de letras en nombres de monedas (sin espacios):**
  - N = 4: 2 monedas (Euro, Real).
  - N = 9: 1 moneda (Yuan Chino).
  - N = 10: 3 monedas (Yen Japones, Rupia India, Franco Suizo).

  - N = 11: 2 monedas (Sol Peruano, Peso Chileno).
  - N = 12: 3 monedas (Peso mexicano, Peso Uruguayo, Libra Egipcia).
  - N = 13: 2 monedas (Peso Argentino, Won Surcoreano).
  - N = 14: 3 monedas (Peso Colombiano, Balboa Panameno, Libra Esterlina).
  - N = 15: 2 monedas (Dolar canadiense, Rand Sudafricano).
  - N = 16: 1 moneda (Dolar Australiano).
  - N = 18: 1 moneda (Colon costarricense).
  - N = 19: 1 moneda (Dolar estadounidense).
- **Ordenamientos manuales:** Algoritmo de burbuja verificado en copias independientes, garantizando la inmutabilidad de los datos originales en memoria.

Validaciones

Se probaron datos inválidos en CSV:

- población `"abc"`;
- área `-5`;
- columna adicional.

El programa muestra una advertencia con el número de línea, omite los datos inválidos y continúa con los registros válidos.

También se verificaron:

- CSV inexistente;
- CSV vacío;
- TOON corrupto;
- entrada inválida del menú;
- letra inválida;
- N inválido.

El TOON corrupto muestra `"No se encontraron datos válidos"` sin provocar que el programa se caiga.

Verificación de reportes TXT, HTML y bitácora (Tarea 6)

Se realizó una auditoría completa de los módulos `reportes.py`, `bitacora.py` y `main.py` mediante una suite de pruebas aislada en entorno controlado:

- **Reporte TXT (`reporte_paises.txt`):**
  - Generación de 133 líneas exactas (25 bloques de 4 líneas + separadores + resumen general).
  - Presencia obligatoria de los 7 campos por país: nombre normalizado, capital, código ISO, población, área, densidad calculada y categoría.
  - Bloque final de resumen general verificado: 25 países, 4 411 741 600 habitantes totales, 62 099 758 km^2 de área total, promedio de población de 176 469 664 hab y promedio de área de 2 483 990.32 km^2.
  - Manejo de error verificado con retorno `False` ante rutas no disponibles.
- **Reporte HTML de Monedas (`reporte_monedas.html`):**
  - Estructura HTML5 válida con `<!DOCTYPE html>`, `<html lang="es">` y `<meta charset="utf-8">`.
  - Tarjeta de estadísticas generales: 21 monedas analizadas, tasa promedio `393.5148`, moneda más fuerte COP (`4150.0000`) y moneda más débil GBP (`0.7600`).
  - Tabla detallada de 21 monedas con colores alternados y formato a 4 decimales.
  - Tabla de clasificación: 16 monedas fuertes, 2 iguales y 3 débiles con sus códigos asociados.
  - Ausencia confirmada de residuos y caracteres extraños.

- **Reporte HTML de Densidad Poblacional (`reporte_densidad.html`):**
  - Estructura HTML5 válida con `<!DOCTYPE html>`, `<html lang="es">` y `<meta charset="utf-8">`.
  - Estadísticas de densidad mundial: promedio aritmético global `127.99 hab/km`^{2}, máxima Corea Del Sur (`516.32 hab/km`^{2}), mínima Australia (`3.34 hab/km`^{2}) y distribución porcentual exacta.
  - Tablas Top 10 mayor y Top 10 menor densidad ordenadas correctamente.
  - Gráfico de barras ASCII vertical renderizado en bloque `<pre>` con marcas de escala numéricas hasta 25 y conteos por columna.
- **Bitácora del Sistema (`bitacora.py` / `bitacora.bin`):**
  - Persistencia binaria en modo append (`"ab"`) con codificación UTF-8 y timestamp exacto de 19 caracteres (`AAAA-MM-DD HH:MM:SS`).
  - Formato por registro: `Fecha_Hora|Descripcion` con soporte para caracteres especiales y separadores múltiples.
  - Manejo de archivo inexistente verificado (retorna lista vacía `[]` sin excepción).
  - Búsqueda por subcadena de fecha y búsqueda por palabra clave verificadas con y sin coincidencias.
  - Exportación a CSV con cabecera `Fecha_Hora;Descripcion` y delimitador `;`.
  - Preservación íntegra de la bitácora real del repositorio (`bitacora.bin`), que se mantuvo intacta con sus 10 registros originales durante todas las pruebas.

Auditoría de integración general y menú principal (Tarea 7)

Se evaluó la integración modular del sistema completo y la robustez del menú principal mediante pruebas automatizadas con subprocesos e inyección de entradas en `stdin`:

- **Conformidad del menú con la consigna:**
  - Presencia exacta de las 9 opciones exigidas por la Sección 5 del enunciado oficial.
  - Cada opción invoca limpiamente la función correspondiente sin lógica huérfana.
  - La Opción 9 finaliza la ejecución con mensaje de despedida y registra la salida en la bitácora.
- **Resiliencia en arranque en frío (independencia de opciones):**
  - Ejecución directa de las opciones 2 a 7 cuando aún no existe el archivo `paises_datos.toon`.
  - El sistema captura de forma segura `FileNotFoundError`, muestra el mensaje amigable `"Primero ejecute la Opción 1"` y regresa al menú sin caerse ni generar excepciones no controladas.
- **Manejo de entradas inválidas:**
  - Menú principal: entradas alfabéticas (`"x"`, `"abc"`), fuera de rango (`"0"`, `"15"`, `"-1"`) y vacías son interceptadas en la cláusula `else`, emitiendo el mensaje `"Opción no válida. Por favor, ingrese un número del 1 al 9."` y registrando el error en la bitácora sin cerrar la sesión.
  - Validaciones internas: validación estricta de una sola letra alfabética en Opción 2 y de números enteros positivos mayores a cero en Opción 4.
  - Submenú de bitácora: captura de opciones fuera del rango A-E y retorno seguro al menú principal con la opción `"E"`.
- **Sesión completa integrada:**
  - Flujo continuo `1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9` ejecutado de extremo a extremo sin errores de memoria ni de tipado.
  - Generación de los archivos de salida (`paises_datos.toon`, `reporte_paises.txt`, `reporte_monedas.html`, `reporte_densidad.html`, `bitacora.bin` y `bitacora.csv`).
- **Inmutabilidad del repositorio real:**
  - Confirmación de que las pruebas en aislamiento no alteraron ni los reportes ni la bitácora original del repositorio, la cual se mantuvo con sus 10 registros históricos originales.

Estilo

Las funciones tienen docstrings y los encabezados del código cumplen con el formato estándar.

Problemas encontrados y soluciones

N.º	Problema	Solución
1	Los cálculos poblacionales debían hacerse manualmente	Se implementaron cálculos mediante las funciones del programa
2	La mediana no podía obtenerse mediante una función automática	Se realizó el ordenamiento manual
3	El criterio de moneda más fuerte generaba una diferencia con la interpretación inicial	Se documentó y verificó el criterio utilizado
4	El CSV con población `"abc"` provocaba un dato inválido	Se agregó validación y se omite la línea incorrecta
5	Un área negativa debía rechazarse	Se agregó validación del área
6	El uso de `round()` estaba prohibido	Se reemplazó por el procesamiento permitido
7	Los cortes de texto estaban prohibidos	Se reemplazaron por procesamiento carácter por carácter
8	Algunas funciones quedaron demasiado largas	Se identificaron para una posible refactorización posterior

Pendiente para los siguientes avances

- Documentación final formal (`DOCUMENTACION_FINAL.md` / `documentaciónCódigos.PDF`).
- Incluir en la documentación portada, índice hipervinculado, referencia al enunciado, 5 olores de software justificados, lista de validaciones estratégicas, estrategia de reportes, reglamento de trabajo en equipo, 2 agendas, 2 minutas, cronograma, 16 problemas y soluciones, 16 lecciones aprendidas y estadísticas de tiempos y esfuerzo PSP.
- Preparar la entrega final vía TEC Digital para el 16 de octubre.

Estado del AVANCE 2

Parte	Estado
Opción 1	Hecha y verificada
Opción 2	Hecha y verificada
Opción 3	Hecha y verificada
Opción 4	Hecha y verificada
Opción 5	Hecha y verificada
Opción 6	Hecha y verificada
Opción 7	Hecha y verificada
Opción 8	Hecha y verificada
Opción 9	Hecha y verificada

Estado general: Completado y verificado al 100%.